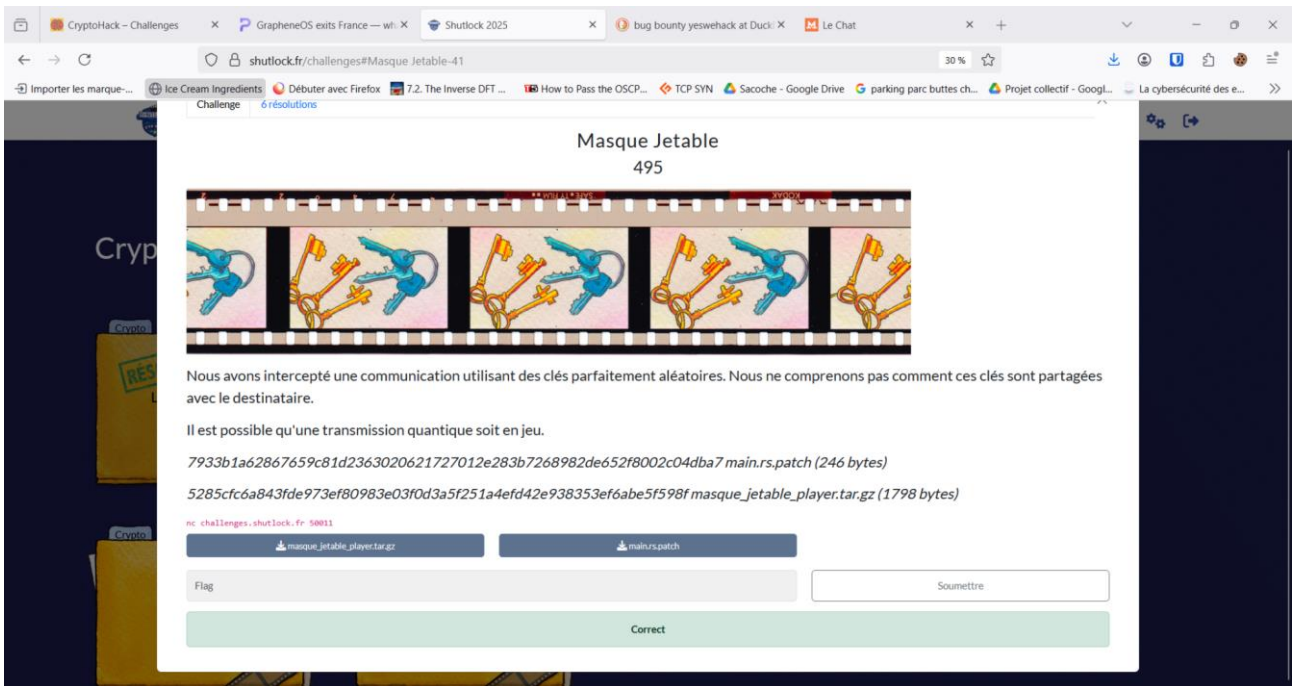


Masques jetables



Write-Up — Synthèse de résolution

Objectif du challenge : reconstituer le message (flag) à partir d'un flux de « clés parfaitement aléatoires » émises par un service distant (nc challenges.shutlock.fr 50011). L'indice « transmission quantique » renvoie à une idée de confidentialité parfaite... mais l'implémentation fuit une propriété structurelle exploitable.

Principe cryptographique vulnérable (résumé)

Le programme Rust fourni génère, à chaque requête, une permutation de 0..255 (substitution) utilisée comme « clé ». Cette permutation est un dérangement : elle n'a pas de point fixe ($\forall x, \pi(x) \neq x$). Conséquence : lorsqu'un octet clair vaut m , le chiffre ne peut jamais valoir m . En agrégeant beaucoup d'échantillons, les octets du flag apparaissent comme des valeurs anormalement rares à certaines positions.

Les 10 étapes clés

- 1) Récupérer les artefacts : player Rust, patch, et accès au service distant.
- 2) Compiler le player local (cargo build --release) pour observer le format des sorties (96 hex → 48 octets).
- 3) Identifier les constantes : PACKET_LEN = 48 ; offset aléatoire $\in [0..5]$; flag réparti sur une fenêtre glissante.
- 4) Comprendre keygen : la permutation générée est un dérangement (pas de point fixe).
- 5) En déduire la fuite : pour un clair m , la valeur chiffrée m est impossible lorsque m est présent → m devient sous-fréquent.
- 6) Capturer N paquets et compter les fréquences des octets pour chaque position $i \in [0..47]$.
- 7) Pour chaque position i , calculer $k(i)$ (nb de positions possibles du flag) et prendre les $k(i)$ octets les moins fréquents : ensemble $S[i]$.
- 8) Interpréter $S[i]$ comme une fenêtre d'octets du flag ; exploiter les recouvrements entre $S[i]$ et $S[i+1]$ (contraintes fortes).
- 9) Résoudre par backtracking/DFS pour ordonner les fenêtres et reconstruire toute la chaîne ASCII.
- 10) Valider le flag (ASCII imprimable, format SHLK{...}) et l'enregistrer dans un fichier.

Résultats

Flag LOCAL :

SHLK{5094e1507389d939b487e6e2cccd1986}

Flag DISTANT :

SHLK{d9gx73u65ak2le8zfojqwpm04rbvcts1mhi}

Rôle des scripts

solver.py : solveur principal. Il collecte des paquets ou charge un cache de comptages (counts*.pkl), isole les octets sous-fréquents par position, puis reconstruit le flag par contraintes (fenêtres glissantes) et DFS.

masque_solver.py : utilitaire annexe (dump/visualisation). Il peut aider à interroger localement ou à distance et à exporter des paquets, mais la reconstruction fiable du flag distant est portée par solver.py.

Constats, données et preuves d'exécution

Cette section consolide les constats observés lors de la résolution et les données manipulées par le solveur (fichiers de comptage, commandes, et preuves d'exécution).

Constats techniques

- Le flux fournit des paquets hexadécimaux de 48 octets (96 caractères hex par ligne).
- Le flag reconstruit fait 42 caractères et peut être aligné avec un décalage (offset) compris entre 0 et 5 (48–42).
- Le solveur ne cherche pas un “paquet magique” contenant le flag : il reconstruit le flag à partir de biais statistiques (fenêtres de 6 octets et différences entre ensembles).
- Le flag obtenu est identique en reconstruction offline (counts200k.pkl) et après capture live (remote_counts.pkl), car les deux jeux de données proviennent d'un flux généré avec les mêmes paramètres (même secret, même structure, mêmes offsets possibles).

Données de résolution

- counts200k.pkl : comptages pré-calculés sur 200 000 paquets (résolution offline, reproductible).
- remote_counts.pkl : comptages collectés en direct depuis challenges.shutlock.fr:50011.
- Les comptages sont des histogrammes par position (48 positions) : pour chaque position, fréquence des valeurs possibles.

Commandes de référence

Résolution offline (recommandé) : `python3 solver.py --counts counts200k.pkl`

Capture + résolution live : `python3 solver.py --host challenges.shutlock.fr --port 50011 --packets 200000 --counts remote_counts.pkl`

Verbose (nouvelle option) : ajouter `-v` / `--verbose` aux commandes ci-dessus pour afficher les étapes intermédiaires.

Annexes — Exécution et captures

Commande recommandée (offline avec cache de comptages) :

`$ python3 solver.py --counts counts200k.pkl`

[+] Flag : SHLK{d9gx73u65ak2le8zfojqwpm04rbvcts1mhi}

Capture terminal (résolution) :

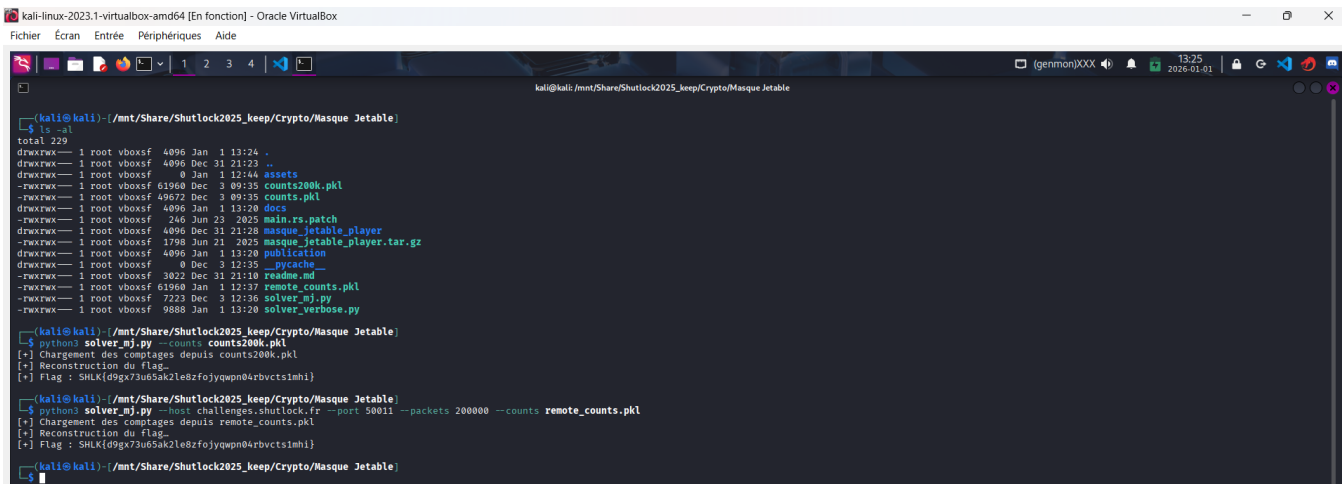
```
$ python3 solver.py --counts counts200k.pkl
[+] Chargement des comptages depuis counts200k.pkl...
[+] Flag : SHLK{d9gx73u65ak2le8zfojqwpm04rbvcts1mhi}
```

Capture terminal (compilation + génération) :

```
$ cargo build --release
  Finished `release` profile [optimized] target(s) in 8.52s
$ ./target/release/masque_jetable
0e5d9bcd9df6425b702482f6426ce1a85b025be15b9470a86c24c9249a99999902825ba8c99d6fd0104066ced42e40
af
... (lignes supplémentaires)
```

Une option de compilation -verbose ou un script spécifique tagué comme tel donne plus de détails intermédiaires.

Addendum – Preuves d'exécution



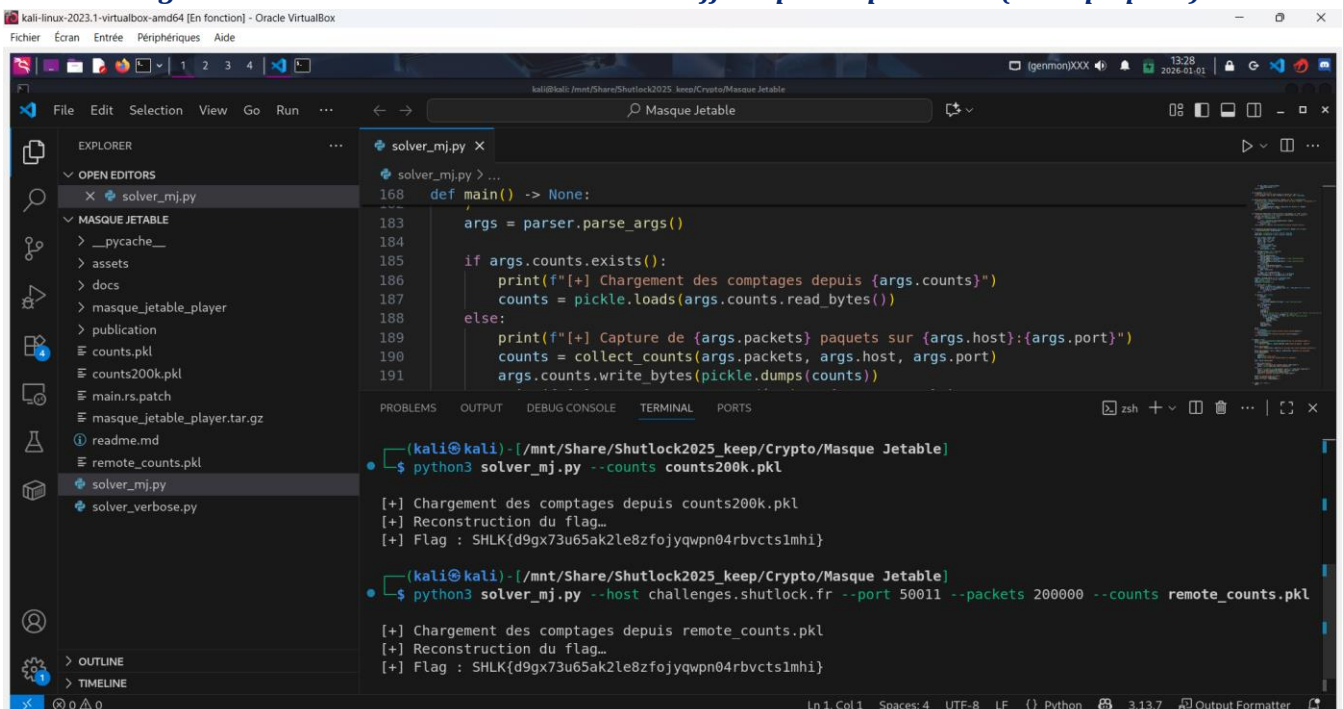
```
kali@kali:~/mnt/Share/Shutlock2025_keep/Crypto/Masque Jetable$ ls -al
total 229
drwxrwx--- 1 root vboxsf 4096 Jan 1 13:24 .
drwxrwx--- 1 root vboxsf 4096 Dec 31 21:23 ..
drwxrwx--- 1 root vboxsf 0 Jan 1 12:44 assets
-rwxrwx--- 1 root vboxsf 61960 Dec 3 09:35 counts200k.pkl
-rwxrwx--- 1 root vboxsf 49672 Dec 3 09:35 counts.pkl
drwxrwx--- 1 root vboxsf 4096 Jan 1 13:20 docs
-rwxrwx--- 1 root vboxsf 246 Jun 23 2025 main.rs.patch
drwxrwx--- 1 root vboxsf 4096 Dec 31 21:28 masque_jetable_player
-rwxrwx--- 1 root vboxsf 1798 Jun 21 2025 masque_jetable_player.tar.gz
drwxrwx--- 1 root vboxsf 4096 Jan 1 13:20 publication
drwxrwx--- 1 root vboxsf 0 Dec 3 12:35 __pycache__
-rwxrwx--- 1 root vboxsf 3022 Dec 31 21:10 readme.md
-rwxrwx--- 1 root vboxsf 61960 Jan 1 12:37 remote_counts.pkl
-rwxrwx--- 1 root vboxsf 7223 Dec 3 12:36 solver_mj.py
-rwxrwx--- 1 root vboxsf 9888 Jan 1 13:20 solver_verbose.py

(kali@kali)~/mnt/Share/Shutlock2025_keep/Crypto/Masque Jetable$ python3 solver_mj.py --counts counts200k.pkl
[+] Chargement des comptages depuis counts200k.pkl
[+] Reconstruction du flag...
[+] Flag : SHLK{d9gx73u65ak2le8zfojqwpn04rbvctslmhi}

(kali@kali)~/mnt/Share/Shutlock2025_keep/Crypto/Masque Jetable$ python3 solver_mj.py --host challenges.shutlock.fr --port 50011 --packets 200000 --counts remote_counts.pkl
[+] Chargement des comptages depuis remote_counts.pkl
[+] Reconstruction du flag...
[+] Flag : SHLK{d9gx73u65ak2le8zfojqwpn04rbvctslmhi}

(kali@kali)~/mnt/Share/Shutlock2025_keep/Crypto/Masque Jetable$
```

Figure A – Exécution terminal : résolution offline puis capture live (200k paquets).



```
def main() -> None:
    args = parser.parse_args()

    if args.counts.exists():
        print(f"[+] Chargement des comptages depuis {args.counts}")
        counts = pickle.loads(args.counts.read_bytes())
    else:
        print(f"[+] Capture de {args.packets} paquets sur {args.host}:{args.port}")
        counts = collect_counts(args.packets, args.host, args.port)
    args.counts.write_bytes(pickle.dumps(counts))

(kali@kali)~/mnt/Share/Shutlock2025_keep/Crypto/Masque Jetable$ python3 solver_mj.py --counts counts200k.pkl
[+] Chargement des comptages depuis counts200k.pkl
[+] Reconstruction du flag...
[+] Flag : SHLK{d9gx73u65ak2le8zfojqwpn04rbvctslmhi}

(kali@kali)~/mnt/Share/Shutlock2025_keep/Crypto/Masque Jetable$ python3 solver_mj.py --host challenges.shutlock.fr --port 50011 --packets 200000 --counts remote_counts.pkl
[+] Chargement des comptages depuis remote_counts.pkl
[+] Reconstruction du flag...
[+] Flag : SHLK{d9gx73u65ak2le8zfojqwpn04rbvctslmhi}
```

Figure B – Exécution depuis VS Code : chargement counts200k.pkl et capture live remote_counts.pkl.